

ESCUELA POLITÉCNICA NACIONAL

Facultad de Ingeniería de Sistemas

Construcción y Evolución de Software



INFORME DE MANTENIMIENTO DE SOFTWARE

CRUD seleccionado: pets-crud

Estudiante: Helen Jarrín

Repositorio base: <https://github.com/2026a-sw-const-gr2/JARRIN-ROMAN-HELEN-SCARLETH-constswgr2.git>

Tema: Mantenimiento correctivo, adaptativo, perfectivo y preventivo aplicado a un CRUD de mascotas

2026-05-29

Índice de contenido

Índice de contenido	2
1. Tema	3
2. Objetivos	3
2.1 Objetivo general	3
2.2 Objetivos específicos	3
3. Alcance y contexto del CRUD	3
4. Diagnóstico inicial	3
5. Desarrollo del mantenimiento realizado	4
5.1 Mantenimiento correctivo	4
5.2 Mantenimiento adaptativo	4
5.3 Mantenimiento perfectivo	4
5.4 Mantenimiento preventivo	5
5.6 Refactorización aplicada	5
6. Evidencias	5
7. Pruebas realizadas	7
8. Clasificación formal de los cambios	7
9. Resultados obtenidos	7
10. Conclusiones	8
11. Recomendaciones	8
12. Bibliografía IEEE	8

1. Tema

Mantenimiento de software aplicado al CRUD pets-crud del repositorio YO-SOY-CRUD, con énfasis en el diagnóstico de deuda técnica, la estabilización funcional, la incorporación de pruebas unitarias, el registro de trazas y la justificación de los cambios según la taxonomía de mantenimiento de software.

2. Objetivos

2.1 Objetivo general

Analizar, corregir y documentar el mantenimiento técnico del CRUD pets-crud, demostrando que la aplicación puede ejecutar operaciones de creación, lectura, actualización y eliminación de mascotas de forma más estable, verificable y mantenible.

2.2 Objetivos específicos

- Identificar la deuda técnica existente en el código base del CRUD seleccionado.
- Aplicar cambios correctivos para evitar errores funcionales y respuestas inconsistentes ante peticiones inválidas.
- Incorporar ajustes adaptativos que permitan configurar puerto, base de datos y conexión con servicios externos mediante variables de entorno.
- Mejorar la observabilidad del sistema mediante logs, endpoint de salud y respuestas de API más claras.
- Agregar controles preventivos, como validaciones, revisión sintáctica y pruebas unitarias automatizadas.
- Clasificar cada intervención según los tipos de mantenimiento: correctivo, adaptativo, perfectivo y preventivo.

3. Alcance y contexto del CRUD

El CRUD pets-crud corresponde a una aplicación Node.js orientada a la gestión de mascotas. El sistema permite registrar información como nombre, especie, raza, edad, propietario, teléfono, estado y descripción. Además, se conecta con un gestor de eventos para reportar acciones importantes como consultas, creación, actualización y eliminación.

La revisión se centró en el archivo principal server.js, el archivo package.json, la estructura de ejecución local y los puntos de integración con la base de datos SQLite y con el Event Manager. El alcance no incluyó rediseño completo de interfaz gráfica, porque el objetivo principal fue estabilizar el backend y evidenciar el mantenimiento del software.

Elemento	Descripción
Lenguaje y entorno	JavaScript ejecutado con Node.js.
Framework web	Express, usado para definir rutas HTTP y servir archivos estáticos.
Base de datos	SQLite mediante node:sqlite, con tabla pets e índices para species, status y owner.
Comunicación externa	Axios para enviar eventos al Event Manager.
Configuración	Variables de entorno: PORT, PETS_DB_PATH, EVENT_MANAGER_URL y EVENT_MANAGER_HEALTH_URL.
Scripts principales	npm start, npm run check, npm run lint y npm test.

4. Diagnóstico inicial

El diagnóstico inicial permitió reconocer puntos de deuda técnica que podían afectar la confiabilidad, el mantenimiento futuro y la demostración del CRUD. La deuda técnica se entendió como trabajo pendiente que no necesariamente impide ejecutar el sistema, pero sí aumenta el riesgo de fallos, dificulta pruebas y complica futuras modificaciones.

Deuda técnica detectada	Descripción	Impacto
-------------------------	-------------	---------

Deuda técnica detectada	Descripción	Impacto
Validación de entrada	Los datos enviados al CRUD requerían controles claros para evitar registros incompletos o inconsistentes.	Riesgo de guardar mascotas con campos obligatorios vacíos, edades inválidas o especies no permitidas.
Manejo de errores HTTP	Las rutas debían responder adecuadamente cuando una mascota no existía o cuando el cuerpo de la petición era inválido.	Riesgo de respuestas ambiguas y dificultad para demostrar peticiones inválidas durante la prueba de vida.
Configuración del entorno	El puerto, la ruta de la base de datos y el servicio de eventos no debían quedar rígidos en el código.	Riesgo de fallos al ejecutar el CRUD en otra máquina o junto con otros servicios.
Observabilidad	El sistema necesitaba evidencias en consola y un endpoint de salud para verificar API, base de datos y conexión externa.	Riesgo de no poder explicar qué sucede internamente al hacer CREATE, QUERY, UPDATE o DELETE.
Pruebas automatizadas	El proyecto requería una suite básica de pruebas para validar funciones críticas sin depender solo de pruebas manuales.	Riesgo de introducir errores nuevos al corregir el CRUD.

5. Desarrollo del mantenimiento realizado

5.1 Mantenimiento correctivo

El mantenimiento correctivo se enfocó en corregir fallos o comportamientos defectuosos. En este caso, el CRUD necesitaba responder correctamente cuando se enviaban datos inválidos o cuando se consultaba, actualizaba o eliminaba una mascota inexistente.

- Se validaron campos obligatorios como name, species y owner.
- Se controló que age sea un número entero dentro de un rango razonable.
- Se limitaron longitudes de campos como name, owner, breed, phone y description.
- Se devolvieron respuestas 400 para datos inválidos y 404 cuando el id de mascota no existía.
- Se evitó que las operaciones PUT y DELETE continuaran si previamente no se encontraba la mascota.

Con estas correcciones, el sistema dejó de aceptar entradas que podían dañar la consistencia de los datos y permitió demostrar de forma clara cómo responde ante peticiones inválidas.

5.2 Mantenimiento adaptativo

El mantenimiento adaptativo permitió que el CRUD se ejecute en distintos entornos sin depender de valores fijos. La intervención consideró que el puerto, la ruta de la base de datos y la URL del Event Manager pueden cambiar entre computadoras, laboratorios o ambientes de prueba.

```
const EVENT_MANAGER_URL = process.env.EVENT_MANAGER_URL || 'http://localhost:3000/events';
const EVENT_MANAGER_HEALTH_URL = process.env.EVENT_MANAGER_HEALTH_URL || 'http://localhost:3000/health';
const PORT = process.env.PORT || 4002;
const DB_PATH = process.env.PETS_DB_PATH || path.join(DB_DIR, 'pets.sqlite');
```

Esta adaptación redujo la dependencia del código fuente respecto al ambiente de ejecución. Por ejemplo, si el Event Manager se ejecuta en otro puerto, solo se cambia la variable de entorno y no el código.

5.3 Mantenimiento perfectivo

El mantenimiento perfectivo buscó mejorar la experiencia de uso, la comprensión del sistema y la demostración. Para ello, se reforzaron elementos que no necesariamente corrigen un error, pero sí hacen que el CRUD sea más útil y fácil de explicar.

- Se incorporó un endpoint /health para comprobar el estado de la API, la base de datos y el Event Manager.
- Se incluyó un endpoint /pets/stats para obtener estadísticas rápidas del total de mascotas y estados relevantes.
- Se generaron mensajes de consola para evidenciar el envío de eventos CREATE, QUERY, UPDATE y DELETE.
- Se mantuvieron respuestas JSON claras para que la prueba con navegador, Postman o curl sea entendible.

Estos cambios facilitan la prueba de vida, porque el evaluador puede observar el funcionamiento del CRUD y, al mismo tiempo, verificar trazas de ejecución en consola.

5.4 Mantenimiento preventivo

El mantenimiento preventivo se aplicó para disminuir la probabilidad de errores futuros. En lugar de depender únicamente de pruebas manuales, el proyecto quedó preparado para ejecutar revisión sintáctica, linting y pruebas unitarias.

- Se expusieron funciones del servidor para que puedan probarse de forma aislada.
- Se utilizó node:test como base para una suite de pruebas unitarias sin necesidad de instalar un framework adicional.
- Se agregó el script npm test para ejecutar los archivos test/*.test.js.
- Se agregó npm run check para validar sintaxis con node --check.
- Se mantuvo npm run lint para detectar problemas de estilo o errores potenciales.

npm	run	check
npm	run	lint
npm test		

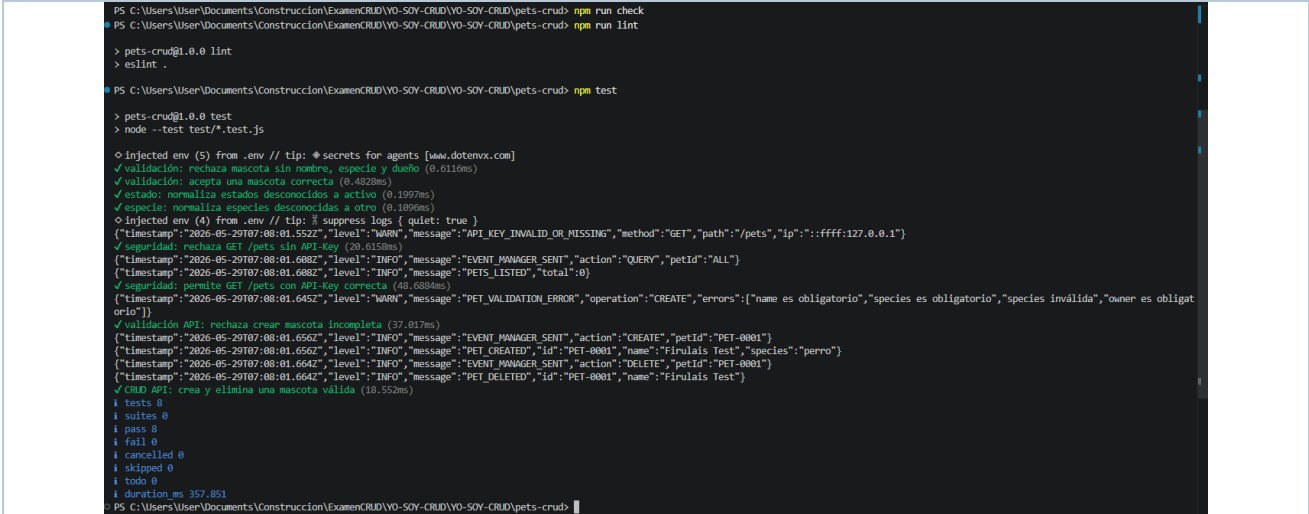
5.6 Refactorización aplicada

La refactorización se realizó separando la lógica en funciones pequeñas y reutilizables. Aunque el archivo principal continúa centralizando la aplicación, ahora contiene funciones específicas para limpiar entradas, normalizar especies, validar mascotas, buscar registros, insertar, actualizar, eliminar y calcular estadísticas. Esto mejora la legibilidad y permite que las pruebas se enfoquen en funciones concretas.

Función	Aporte al mantenimiento
clean()	Estandariza la limpieza de datos recibidos.
normalizeSpecies()	Evita especies no permitidas y asigna un valor controlado.
normalizeStatus()	Controla el estado de la mascota.
validatePet()	Agrupar reglas de validación para creación y actualización.
findPetById()	Centraliza la búsqueda por id.
insertPet(), updatePet(), deletePetById()	Separan la lógica de persistencia de las rutas HTTP.
getPetStats()	Resume información estadística para la API.

6. Evidencias

Captura 1. Revisión sintáctica del servidor



Se muestra npm run check, lint y test. Esta evidencia confirma que el archivo principal no presenta errores de sintaxis antes de ejecutar el sistema.

[illegible]

Se observa el mensaje de consola que indica que el sistema de gestión de mascotas está corriendo en el puerto configurado y que la base de datos fue localizada y se muestran los logs.

[illegible]

Captura 4. Creación de mascota con POST /pets fALLIDO

Se muestra una petición con datos incompletos o inválidos y una respuesta 400. Esta captura respalda el mantenimiento correctivo.

7. Pruebas realizadas

Las pruebas se organizaron en dos niveles: pruebas manuales de API y pruebas automatizadas. Las pruebas manuales validan el comportamiento visible del CRUD; las pruebas automatizadas reducen el riesgo de regresiones futuras.

Prueba manual	Propósito	Resultado esperado
GET /health	Verificar estado de API, base de datos y Event Manager.	Debe retornar JSON con status ok y datos de diagnóstico.
GET /pets	Listar todas las mascotas registradas.	Debe retornar un arreglo JSON.
POST /pets	Crear mascota con datos válidos.	Debe retornar 201 y un objeto con id PET-####.
POST /pets inválido	Enviar name, species u owner vacío.	Debe retornar 400 con mensaje de error claro.
GET /pets/:id inexistente	Consultar un id que no existe.	Debe retornar 404 y mensaje Mascota no encontrada.
PUT /pets/:id	Actualizar una mascota existente.	Debe retornar el objeto actualizado.
DELETE /pets/:id	Eliminar una mascota existente.	Debe retornar mensaje de eliminación y el objeto eliminado.

8. Clasificación formal de los cambios

Cambio aplicado	Tipo de mantenimiento	Justificación
Validación de campos obligatorios y rangos	Correctivo / Preventivo	Corrige entradas inválidas actuales y evita que en el futuro se guarden datos inconsistentes.
Respuestas 400 y 404 en rutas POST, PUT, GET y DELETE	Correctivo	Soluciona comportamientos ambiguos ante peticiones incorrectas o recursos inexistentes.
Variables de entorno para puerto, base de datos y Event Manager	Adaptativo	Permite ejecutar el CRUD en distintos entornos sin modificar el código fuente.
Endpoint /health	Perfectivo	Mejora la capacidad de diagnóstico y facilita la prueba de vida.
Endpoint /pets/stats	Perfectivo	Agrega valor al sistema mostrando métricas útiles del dominio de mascotas.
Logs de envío de eventos	Perfectivo / Preventivo	Facilita depuración durante la ejecución y permite detectar fallos de integración.
Exportación de funciones y pruebas con node:test	Preventivo	Reduce el riesgo de regresiones y permite verificar funciones críticas de forma repetible.
Scripts npm run check, npm run lint y npm test	Preventivo	Formaliza controles de calidad antes de entregar o subir cambios.

9. Resultados obtenidos

Después de la intervención, el CRUD pets-crud quedó en mejores condiciones para ser ejecutado, probado y sustentado. El sistema puede crear, consultar, actualizar y eliminar mascotas, pero también puede responder de manera controlada ante datos inválidos o recursos inexistentes.

- Se logró identificar deuda técnica relacionada con validación, manejo de errores, configuración, observabilidad y pruebas.
- Se estabilizaron rutas principales del CRUD mediante validaciones y respuestas HTTP más precisas.
- Se mejoró la adaptabilidad del proyecto al permitir configuración mediante variables de entorno.
- Se incorporaron evidencias de ejecución mediante logs, endpoint /health y endpoint /pets/stats.
- Se dejó una base de pruebas automatizadas y comandos de control para facilitar mantenimiento futuro.
- Se preparó una explicación formal que relaciona cada cambio con la taxonomía de mantenimiento de software.

10. Conclusiones

- Se logró aplicar mantenimiento de software sobre el CRUD pets-crud de manera organizada, partiendo de un diagnóstico técnico y terminando con una intervención justificable y demostrable.
- La aplicación de validaciones y respuestas HTTP controladas permitió mejorar la confiabilidad del CRUD, especialmente ante peticiones inválidas o ids inexistentes.
- La configuración mediante variables de entorno fortaleció la capacidad del sistema para ejecutarse en distintos ambientes sin modificar directamente el código fuente.
- La incorporación de logs, endpoint de salud y estadísticas facilitó la observabilidad del sistema durante la prueba de vida.
- El uso de pruebas, revisión sintáctica y linting permitió reforzar el mantenimiento preventivo, porque el proyecto queda mejor preparado para detectar errores antes de futuras entregas.
- La clasificación de cambios permitió evidenciar que el mantenimiento no fue únicamente corrección de errores, sino también adaptación, mejora funcional y prevención de fallos futuros.

11. Recomendaciones

- Mantener las pruebas automatizadas actualizadas cada vez que se agregue una nueva ruta o regla de negocio.
- Separar progresivamente el archivo server.js en carpetas como routes, controllers, services y repositories si el CRUD continúa creciendo.
- Agregar un archivo .env.example para documentar las variables de entorno necesarias sin exponer información sensible.
- Usar Postman o una colección equivalente para guardar las peticiones de demostración del CRUD.

12. Bibliografía IEEE

- [1] OpenJS Foundation, “Express routing,” Express.js Documentation. [En línea]. Disponible: <https://expressjs.com/en/guide/routing.html>. [Accedido: 02-jun-2026].
- [2] Node.js Project, “Test runner,” Node.js Documentation. [En línea]. Disponible: <https://nodejs.org/api/test.html>. [Accedido: 02-jun-2026].
- [3] SQLite Consortium, “SQLite Documentation,” SQLite. [En línea]. Disponible: <https://www.sqlite.org/docs.html>. [Accedido: 02-jun-2026].
- [4] Git Project, “Git Documentation,” Git. [En línea]. Disponible: <https://git-scm.com/doc>. [Accedido: 02-jun-2026].
- [5] J. Guaman, “YO-SOY-CRUD,” GitHub Repository. [En línea]. Disponible: <https://github.com/JonathanGuaman2004/YO-SOY-CRUD.git>. [Accedido: 02-jun-2026].